

COLORADO STATE UNIVERSITY PUEBLO

School of Engineering

A Graph Neural Surrogate for Longitudinal Train Dynamics

Toward Fuel-Optimal Control of Freight Consists

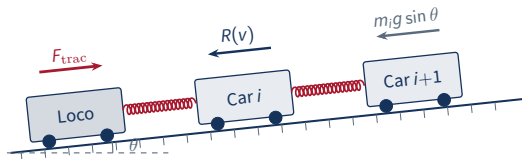
George James

Master's Thesis Defense • September 23, 2026



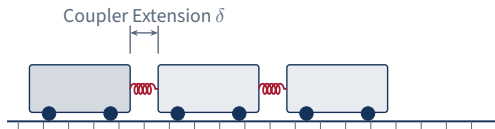
Background: What Longitudinal Train Dynamics Are

- A freight train is a **distributed** mechanical system: tens to hundreds of coupled masses.
- Each vehicle feels the grade, curvature and resistance at its **own** position on the route.
- Couplers carry **slack**: free travel before any force is transmitted.
- Quantities that matter in operation: **fuel**, **arrival time**, **peak coupler force**.



$$m_i \dot{v}_i = F_{\text{ext},i} + F_{i-1} - F_i$$

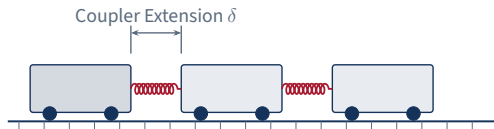
Coupler Slack Makes the Problem Hard



Bunched: Slack Closed, No Force Transmitted



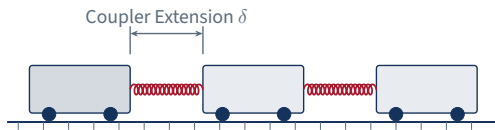
Coupler Slack Makes the Problem Hard



Slack Taken Up: Couplers Reach the End of the Slack



Coupler Slack Makes the Problem Hard



Run-Out: Force Spikes When the Slack Closes



When slack closes suddenly, the force spike can break a coupler. The same **abrupt change in force** makes the numerical solver take very small time steps, which is why the simulation is slow.

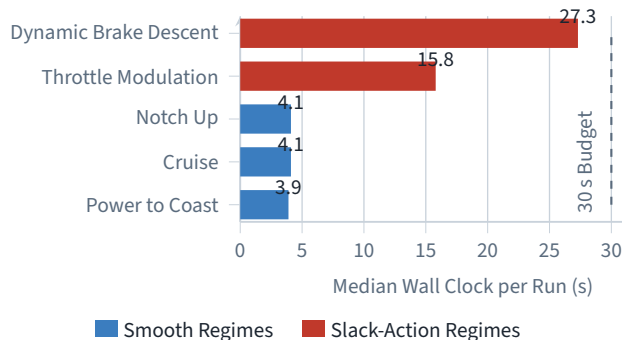
High-Fidelity Simulation Is Too Slow to Use in a Loop

Task	Reported Cost	Source
Whole-trip LTD optimization, sequential	~18 months	Wu et al. (2016)
the same study, run in parallel	11 days	Wu et al. (2016)
Multibody vehicle-track simulation, 5 km	~30 minutes	Zhou et al. (2023)
learned surrogate, same 5 km	~8 seconds	Zhou et al. (2023)

At this cost, any task that needs thousands of simulations is impractical: optimizing a driving plan, testing many scenarios, or running a controller.

Figures quoted as reported; not normalized to a common hardware baseline.

Our Reference Simulator Has the Same Problem



Runs completed within the 30-second budget, by consist size:

16 / 22

$N = 11$ median 4.0 s

4 / 22

$N = 60$ median 28.1 s

0 / 22

$N = 130$ median > 30 s

The slowest runs are the ones with frequent slack events, because each event makes the solver shrink its step. These runs also produce the largest coupler forces, which are the forces a safety limit is meant to control.

Surrogate Modeling



A surrogate is a fast model trained on simulator output. It gives up a small, measured amount of accuracy for a large gain in speed. It **does not replace** the simulator: the simulator produces the training data and remains the reference the surrogate is checked against.

Can a train-agnostic, differentiable surrogate of longitudinal train dynamics be accurate enough, and fast enough, to sit inside a fuel-optimal control loop?

- 1 A **graph representation** of the train, with vehicles as nodes and couplers as edges, that works for any train length.
- 2 A **GPU version of the reference simulator** that runs many scenarios in parallel, and two 10,000-scenario datasets built with it.
- 3 A **graph network simulator** that advances the train by one learned 0.25 s step, replacing 50 evaluations of the physics equations. It is compared against simple baselines, an MLP and a transformer.
- 4 A **fuel-optimal control demonstration** (MPC) that uses the surrogate's gradients to improve the driving plan.

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

10. Control

11. Conclusion

The talk follows the thesis in order.

Acts 3–6 cover the formulation, the reference simulator and the dataset. These parts are complete and validated.

Acts 7–10 are the surrogate itself, its evaluation, and the control demonstration it enables.

This strip appears at the start of each act to show where we are.

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

10. Control

11. Conclusion

ACT 2

Related Work

Four Literatures, One Gap

Classical LTD Simulation

Physics-anchored, train-agnostic,
far too slow to optimize through

Data-Driven Rail Models

Fast, but tied to one consist
or one corridor

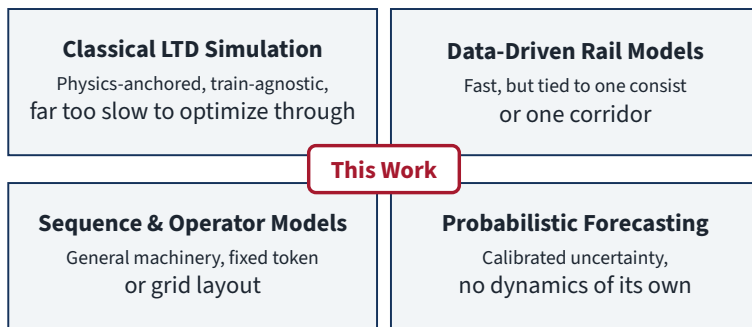
Sequence & Operator Models

General machinery, fixed token
or grid layout

Probabilistic Forecasting

Calibrated uncertainty,
no dynamics of its own

Four Literatures, One Gap



Each area solves part of the problem. None provides a model that is fast, built on the physics, *and* usable for control on a train it was not trained on.

Classical Longitudinal Train Dynamics Simulation

Tool	Availability
TrainDy	Consortium
TOES	Commercial (AAR)
TEDS	Publicly documented
UM Train	Commercial
TDEAS	Research / institutional
CRE-LTS	Research / in-house

Representative LTD tools; see Ch. 2 for full scope and origin.

- Fidelity ranges from a single point mass to full constrained multibody dynamics.
- Most tools were developed as **in-house software**, with little published detail on numerical methods or assumptions (Spiryagin et al., 2017).
- Computing coupler forces takes about **65% of solver run time** (Wu & Cole, 2015). The slack is the most expensive part to compute.

Data-Driven and Hybrid Modeling in Rail

Approach	What It Predicts	What It Is Fixed To
Surrogate elements inside MBD (Nie et al., 2018)	Component response	One mechanical element, one model
RNN surrogate as an FMU (Zhou et al., 2023)	Nominal simulator output	One track segment
CNN + attention force estimation (Zhang et al., 2024)	Coupler force history	One coupler, one consist
In-trainNet (2025)	Forces at several couplers	Still needs per-configuration adaptation data
Rescheduling surrogates (Liu et al., 2024)	Schedule quality	One operational context

The third column is the key point. Each of these models is fast, but each is **tied to one specific** consist, coupler position or track section, and must be retrained when that changes.

Sequence Models and Neural Operators

Long-Horizon Forecasting

Informer, Autoformer, Temporal Fusion Transformer: attention models that handle long time series and combine fixed and time-varying inputs.

Operator Learning

FNO learns the solution of a differential equation in a form that works at any grid resolution. PINO adds a penalty when predictions violate the governing equations.

Why these models are a natural first choice, and why their structure does not fit this problem.

- A transformer takes a **fixed-length input**: an 11-car and a 150-car train must both be padded to the same length.
- The goal is a model that works for any train length. A fixed input length works against that goal.
- PINO uses physics only as a *training penalty*. In this work the physics is *built into the model's calculation*.

Graph Networks for Physical Systems



Vehicles Are Nodes, Couplers Are Edges

The Chain Is the Consist: Any Length, No Padding

This model comes from work on message passing for particle and mesh simulations:
Interaction Networks (Battaglia et al., 2016), Graph Network Simulators
(Sanchez-Gonzalez et al., 2020), and MeshGraphNets (Pfaff et al., 2021).

Graph Networks for Physical Systems

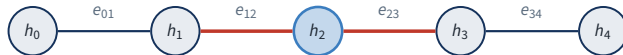


Edge Update

$$e_{ij} \leftarrow \phi_e(e_{ij}, h_i, h_j)$$

Each edge (coupler) is updated from the two vehicles it connects. This mirrors the coupler force law $F_i = \phi_i(\delta_i, \dot{\delta}_i)$.

Graph Networks for Physical Systems

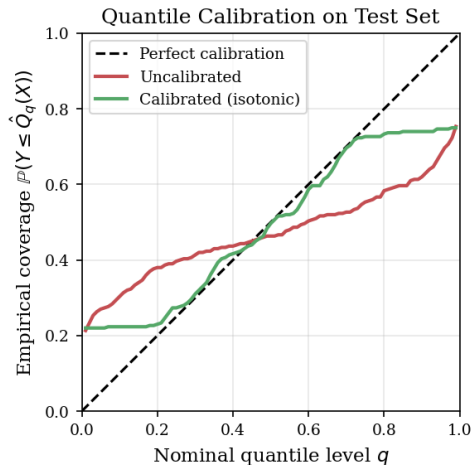


Node Update

$$h_i \leftarrow \phi_h(h_i, \sum_{j \in \mathcal{N}(i)} e_{ij})$$

Each node (vehicle) is then updated from the edges attached to it. This mirrors the force balance $m_i \dot{v}_i = F_{\text{ext},i} + F_{i-1} - F_i$. **The number of model parameters does not depend on N .**

Probabilistic Forecasting and Its Reduced Role



The diagonal is perfect calibration.

- The original plan was to predict probability distributions: quantiles and the chance of exceeding a limit.
- Calibration (honest probabilities) trades against **sharpness** (narrow ranges). Deep models are often overconfident on unfamiliar data.
- This thesis narrows that scope. Uncertainty is used only as a **safety margin** the controller must respect.

The Gap, in One Table

Approach	Train-Agnostic	Physics-Anchored	Differentiable for Control
Classical LTD simulators	✓	✓	×
In-train force estimators	×	×	✓
Surrogate replacement inside MBD	×	~	×
Sequence and operator surrogates	×	~	✓
This work	✓	✓	✓

No existing rail surrogate is train-agnostic, built on the physics, and differentiable for control at the same time. **Each existing approach meets at most two of the three.**

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

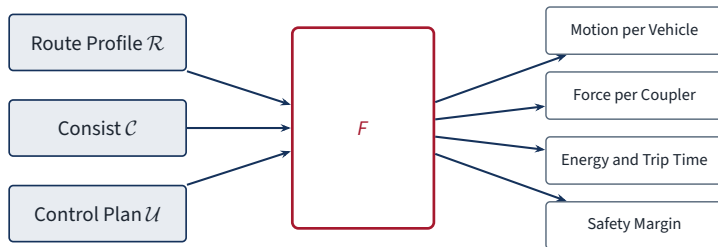
10. Control

11. Conclusion

ACT 3

Problem Formulation

The Prediction Task

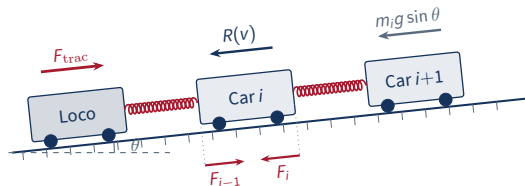


A forecast answers *what will happen* under one driving plan. An optimizer needs to know **how the outcome changes when the plan changes**, and it asks that question thousands of times.

Classical Longitudinal Formulation

- N vehicles indexed lead to tail, $N - 1$ couplers between them.
- Coupler forces come from a force law $F_i = \phi_i(\delta_i, \dot{\delta}_i)$. It includes the slack and the different stiffness in compression (buff) and tension (draft).
- End conditions $F_{-1} = F_{N-1} = 0$.

$$\begin{aligned}\dot{x}_i &= v_i \\ m_i \dot{v}_i &= F_{\text{ext},i} + F_{i-1} - F_i\end{aligned}$$



These equations are standard. The contribution is how they are organized for learning.

Why a Graph, Not a Sequence

Sequence Model: Fixed Input Length



11-car train: 3 tokens carry data, 27 slots are padding



150-car train: the budget is exactly full

The budget must be set for the longest train,
so every short one wastes most of the tensor

The chain shape is already in the equations: each coupler depends only on the two vehicles adjacent to it.

Why a Graph, Not a Sequence

Graph Model: The Chain Is the Consist



short consist: 4 nodes, 3 edges



long consist: same weights

No padding, no masking. The parameter count
does not depend on N

Because the model works on the graph directly, it **handles any train length without padding**. Train length is the variable this thesis aims to generalize over.

The Node Tensor $\mathbf{H}$: One Row per Vehicle

	x	v	z_{ork}	z_{trac}	m_{ass}	$D_{\text{avis A}}$	$D_{\text{avis B}}$	$D_{\text{avis C}}$	$T_{\text{raction?}}$	$f_{\text{max trac}}$	$f_{\text{max brk}}$
Vehicle 0											
Vehicle 1											
Vehicle 2											
Vehicle 3											
Vehicle 4											
$\vdots$											

4 Dynamic 7 Static: Stored Once, Not per Time Step

$\mathbf{H}(t) \in \mathbb{R}^{N \times d_n}$. Four channels change over time; seven are constants for each vehicle. The **constant columns are stored once**, not at every time step. This choice matters again in Act VI.

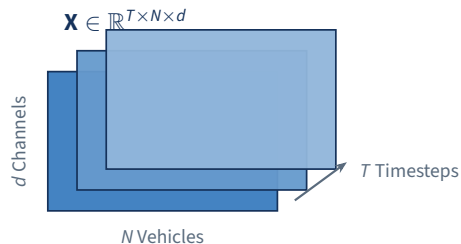
The Edge Tensor E: One Row per Coupler

	δ	$\dot{\delta}$	F	L ₀	Slack	k _{draft}	c _{draft}	k _{buff}	C _{buff}
Coupler 0									
Coupler 1									
Coupler 2									
Coupler 3									
⋮									

3 Dynamic 6 Static

$\mathbf{E}(t) \in \mathbb{R}^{(N-1) \times d_e}$ can be computed directly from the node state: $\mathbf{E} = \mathcal{G}_e(\mathbf{H}, \mathbf{p})$. Only $\mathbf{H}$ holds quantities that must be integrated over time. **E is stored anyway**, and Act VII (slide 69) shows why.

The Rollout Tensor



- Simulator output, training target and surrogate prediction all share **one layout**.
- Coupler histories stack the same way, $\mathbf{E}_{\text{hist}} \in \mathbb{R}^{T \times (N-1) \times d_e}$.
- Because the layout is shared, no conversion code is needed between the simulator and the model.

$$\frac{d}{dt} \text{vec}(\mathbf{H}_{\text{dyn}}(t)) = \mathcal{F}_n(\mathbf{H}, \mathbf{E}, \mathcal{U}, \mathcal{R}; t)$$
$$\mathbf{E}(t) = \mathcal{G}_e(\mathbf{H}(t), \mathbf{p})$$

This rearranges the state and parameters. It does **not** change the force balances.

Possible extension: modeling the internal state of the draft gear would add equations $\mathcal{F}_e$ for the edge rows. This is left for future work.

What the Surrogate Predicts

State Channel

x : position

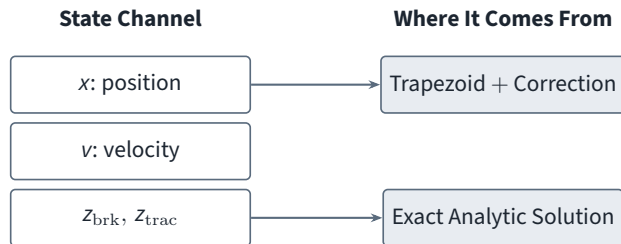
v : velocity

$z_{\text{brk}}, z_{\text{trac}}$

Where It Comes From

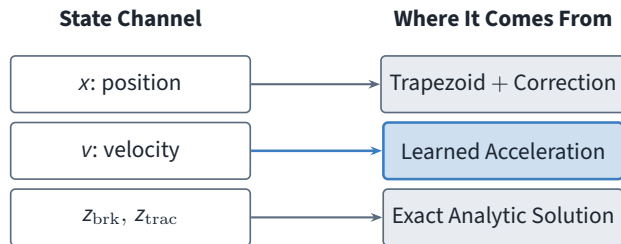
The surrogate does not learn a correction to the physics equations. It learns **one fixed time step** of the same tensor state, from s_k to s_{k+1} over 0.25 s.

What the Surrogate Predicts



Position is computed from velocity with the trapezoid rule. The brake and traction lags use the **exact** solution of $\dot{z} = (u - z)/\tau$. Neither is learned.

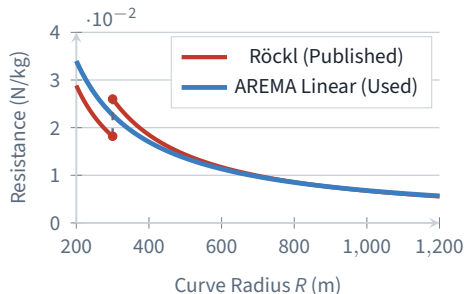
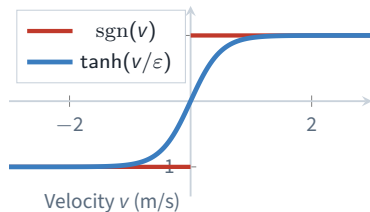
What the Surrogate Predicts



The network outputs two scalars per vehicle,
not four channels

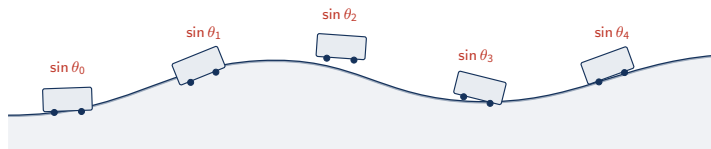
This design imposes more physics than a correction-based design, not less. Two of the four channels are removed from the network entirely. Act VII gives the measurements behind this split.

The Model Must Be Differentiable



The controller uses the model to *choose* the commands $\mathcal{U}$, so the model must provide ∂ outputs/ $\partial \mathcal{U}$: how each output changes when a command changes. The simulator is kept **smooth for a related reason**: a jump in its equations becomes a jump in the training data that the learned step would have to reproduce. The brake term uses a smooth tanh blend instead of a sign function, and the smooth AREMA curvature law replaces the Röckl formula, which jumps at $R = 300$ m.

Route Fields Are Sampled per Vehicle



Every vehicle samples the route at its own chainage $x_i(t)$.
A 2 km consist is on several grade features at once

Grade $\sin \theta(s)$, curvature $\kappa(s)$ and speed limit $v_{\max}(s)$ are defined along the track distance s and evaluated at $s = x_i(t)$ for **each vehicle separately**. For this reason, per-vehicle route sampling is a separate stage of the reference simulator.

Outputs, Objective, and Evaluation Targets

Trajectory Level

Position, velocity and
acceleration per vehicle
Force at every coupler

Aggregate

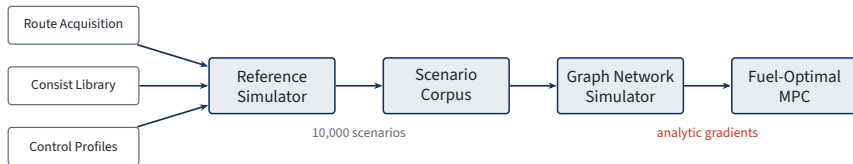
Trip energy
Trip time and distance
travelled

Risk

Peak coupler force
Margin to the operating limit

Trip-level accuracy is **evaluated directly**, not assumed from trajectory accuracy. Small errors at each step can add up, and a controller needs the correct trip energy to compare driving plans.

The Pipeline, End to End



The rest of the talk walks this left to right: the reference simulator (Act IV), the routes feeding it (Act V), the corpus it produced (Act VI), the surrogate trained on that corpus (Acts VII–IX), and the controller it enables (Act X).

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

10. Control

11. Conclusion

ACT 4

Reference Simulator

Built One Physical Effect at a Time

Stage 1 Single Powered Vehicle

Stage 2 Two Bodies, Linear Coupler

Stage 3 Nonlinear Coupler with Slack

Stage 4 Three Bodies, Force Propagation

Stage 5 General N -Vehicle Assembly

Stage 6 Per-Vehicle Grade and Curvature

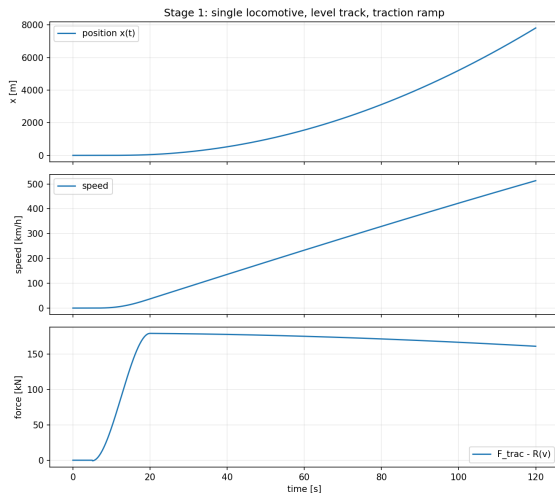
Stage 7 Actuator Lag, Power-Limited Traction

Stage 8 Scope vs Industrial High Fidelity

- Each stage adds **one** physical effect and is tested on a case with a known expected result.
- All later results are **measured against this simulator**, so it must be correct.
- All nine stage figures can be regenerated from the notebook and match the originals **pixel for pixel**.

Stage 1: Single Powered Vehicle

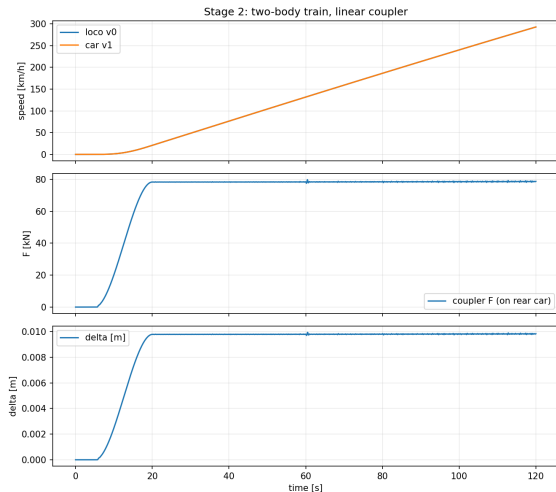
1 2 3 4 5 6 7 8



Traction against rolling and air resistance (the Davis equation) on level track. Net force falls to zero as the vehicle reaches a steady speed. This confirms the integrator and the force signs before couplers are added.

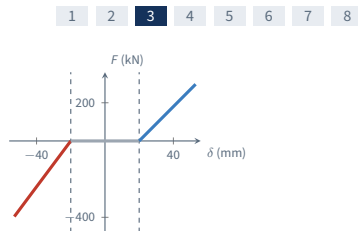
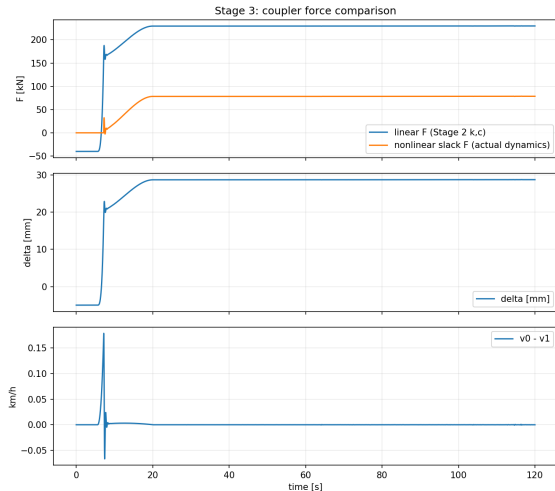
Stage 2: Two Bodies, Linear Coupler

1 2 3 4 5 6 7 8



The two speed traces overlap at this scale.
The coupler force and extension show how the two vehicles interact.

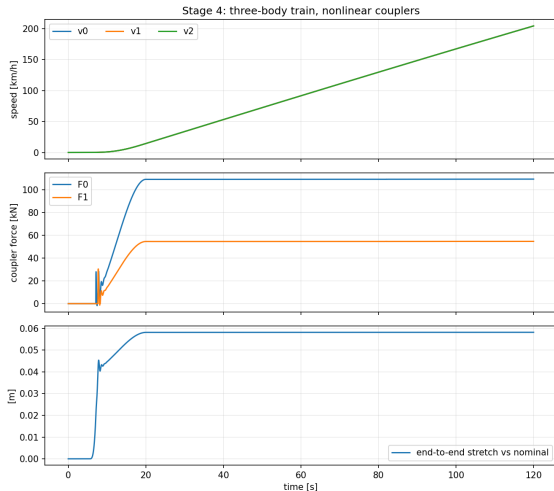
Stage 3: Nonlinear Coupler with Slack



The coupler moves freely within ± 20 mm (the slack) before it transmits force, and is stiffer in one direction than the other. Shown against the Stage 2 linear coupler on the same run. **The force now changes abruptly**, which is what slows the solver.

Stage 4: Three Bodies and Force Propagation

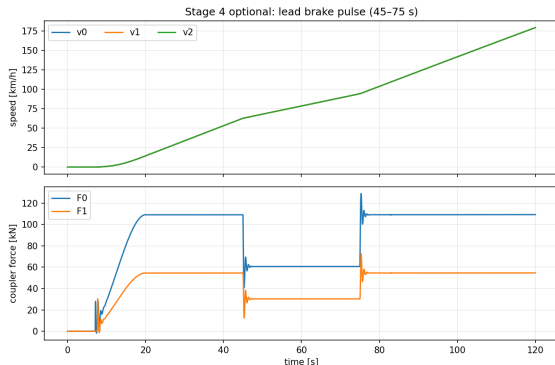
1 2 3 4 5 6 7 8



Force travels down the chain one coupler at a time. End-to-end stretch is computed from the vehicle positions and previews how a long train behaves.

Stage 4b: Lead-Only Brake Pulse

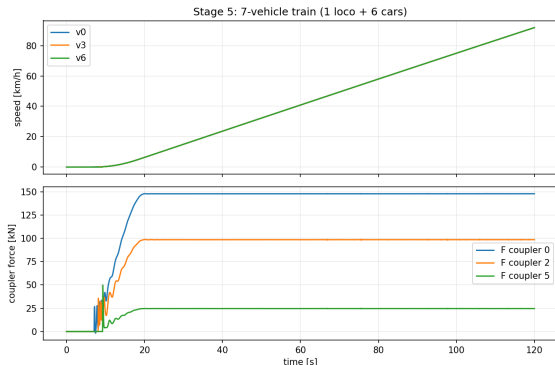
1 2 3 4 5 6 7 8



The run-in case: braking at the head end pushes the chain together. Compare the sign of the coupler forces against the previous slide.

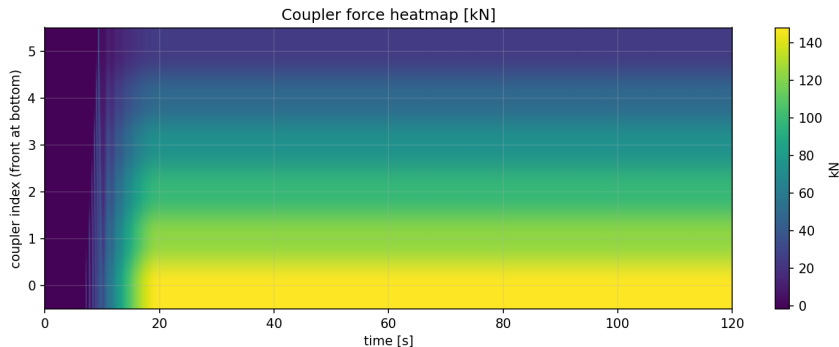
Stage 5: General N -Vehicle Assembly

1 2 3 4 **5** 6 7 8



The generalization from three bodies to arbitrary N , with no per- N special-casing anywhere in the force assembly.

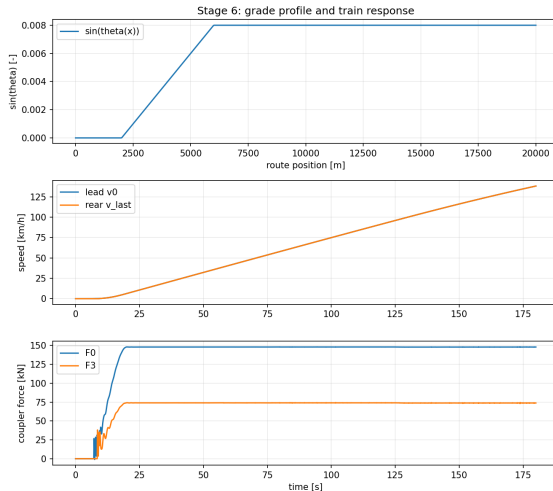
Stage 5: The Coupler-Force Heatmap



Force against time and coupler index. **Force decreases from front to rear** because the lead coupler pulls the entire trailing mass and the last coupler pulls a single car. The narrow band before $t \approx 20$ s is the rising traction force travelling back through the slack.

Stage 6: Per-Vehicle Grade and Curvature

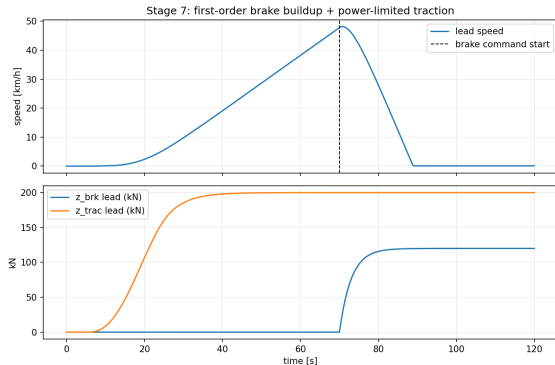
1 2 3 4 5 6 7 8



Each vehicle reads the route at its own position. Lead and rear speeds differ wherever the train spans a change in grade, as described on slide 25.

Stage 7: Actuator Lag and Power-Limited Traction

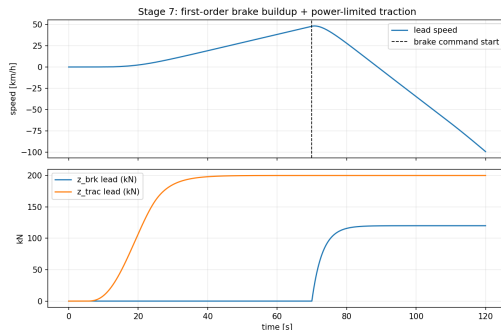
1 2 3 4 5 6 **7** 8



First-order lag states for brake and traction, with tractive effort capped by available power at speed. A driving command changes these lag states, which then change the applied force.

A Brake Sign Error and Its Fix

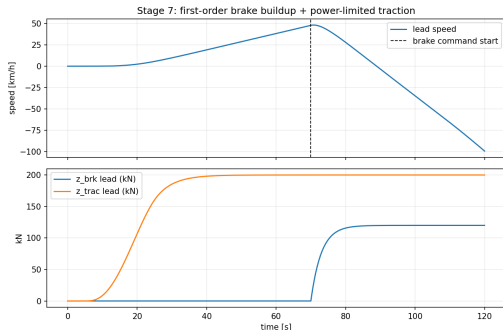
As Published, Before 2026-08-17



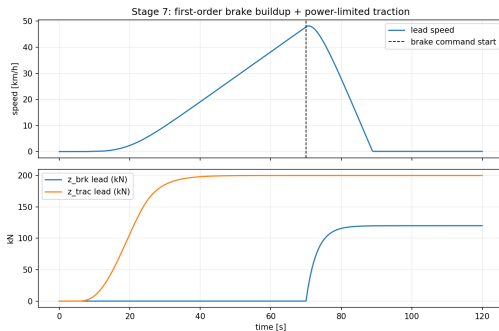
Brake force always pushed in the $-x$ direction, regardless of which way the train was moving. The speed passes through zero and keeps falling to -99 km/h: a train moving backward at 99 km/h under full braking.

A Brake Sign Error and Its Fix

As Published, Before 2026-08-17



Corrected



The fix makes brake force oppose the direction of motion, using a smooth $\tanh(v/\varepsilon)$ blend instead of $\text{sgn}(v)$. The error went unnoticed because all earlier scenarios started from rest under traction. **Randomized starting conditions exposed it.** The fix changed **only one** of the nine stage figures.

What the Simulator Is, and Is Not

In Scope

- Longitudinal motion per vehicle
- Nonlinear couplers with slack
- Asymmetric buff and draft stiffness
- Per-vehicle grade and curvature
- First-order actuator lag
- Power-limited traction

Out of Scope

- Lateral and vertical dynamics
- Wheel-rail contact mechanics
- Draft-gear internal states
- Air-brake pipe propagation
- Hysteresis and stick-slip
- Calibration to real rolling stock

Parameters are **realistic but not calibrated** to any specific locomotive or railcar. All results that follow measure agreement *with this simulator*, not with real trains.

220

Tests Passing

135 pre-existing • 60 torch
acceptance
10 dataset • 14 curvature

- `test_brake_direction`: brakes never accelerate a stopped train, and brake force changes smoothly through zero speed. A hard $\text{sgn}(v)$ fails the second check.
- `test_tensorized_matches_classical`: the two implementations of the equations agree, with the brake option on and off.
- `test_roeckl_is_discontinuous_at_300m`: confirms the jump in the *published formula*, so the test fails if the formula is changed by mistake.

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

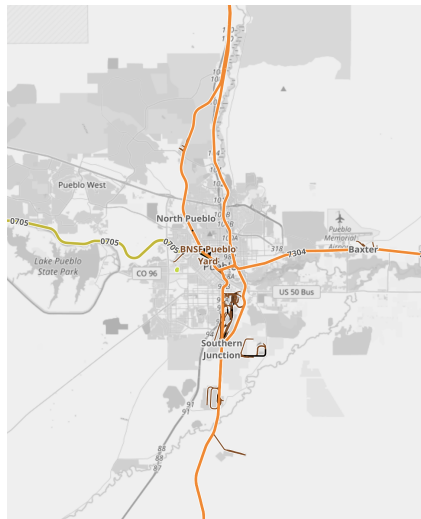
10. Control

11. Conclusion

ACT 5

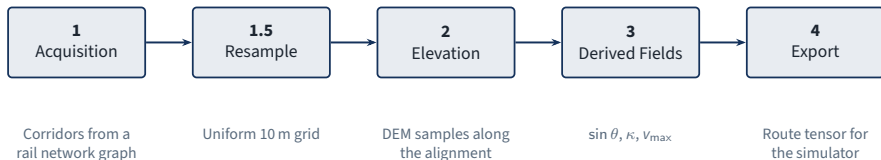
Routes and Terrain

Rail Corridors Must Be Rebuilt from Map Data



- Mainline corridors (orange) run through a dense tangle of yard and service track (brown), meeting at junctions.
- A corridor is not stored as one map object. It must be **assembled from many short segments**.
- Routes in this thesis come from **real elevation and alignment data**, so grades and curves resemble an actual rail corridor rather than an invented profile.

The Pipeline, in Five Stages



Each stage is checked before the next is added, as with the simulator. Once an error in the grade reaches the simulator, it cannot be told apart from real terrain.

Stage 1: Corridor Extraction

Acquire — fetch routes from OSM

Overpass endpoint

If one endpoint returns 406/429, switch to another.

Pulls every track way intersecting the ball, un-clipped. Default centre: Pueblo, CO.

Selection mode
☒ Radius (around point)
☐ Bounding box
☐ Relation ID

Centre latitude: - +
Centre longitude: - +
Radius (km):

Railway types
☒ rail ☒ narrow_gauge ☐

Min corridor (km):

☒ Main lines only (exclude yards/spurs)

☐ Follow lines beyond radius (bounded crawl) ?

Fetch radius & extract corridors

Map data is downloaded for an area first. Corridors are then recovered: yards and spurs are filtered out, the search expands outward up to a limit, fragments are joined, and corridors below a minimum length are discarded.

Stage 1.5: Resampling to a Uniform Grid

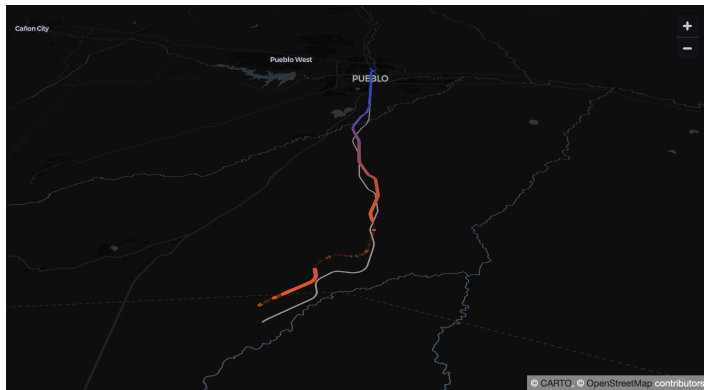


A 49.1 km corridor rewritten onto a uniform **10 m grid**: 4,920 points.

713 of them are interpolated across long source segments rather than sitting on a mapped vertex.

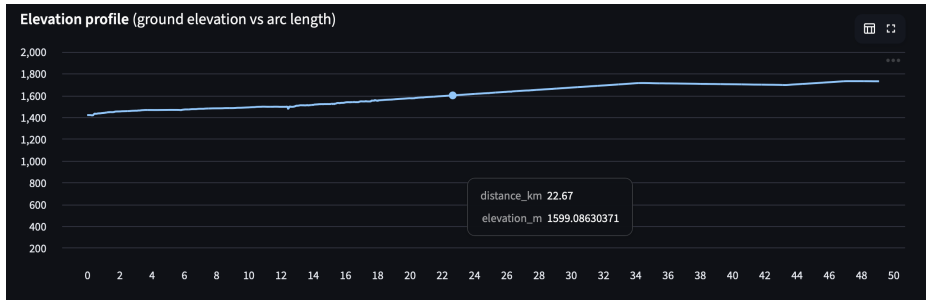
These points stay flagged in later stages, so it is always clear where the geometry is estimated rather than mapped.

Stage 2: Elevation, and Looking at It



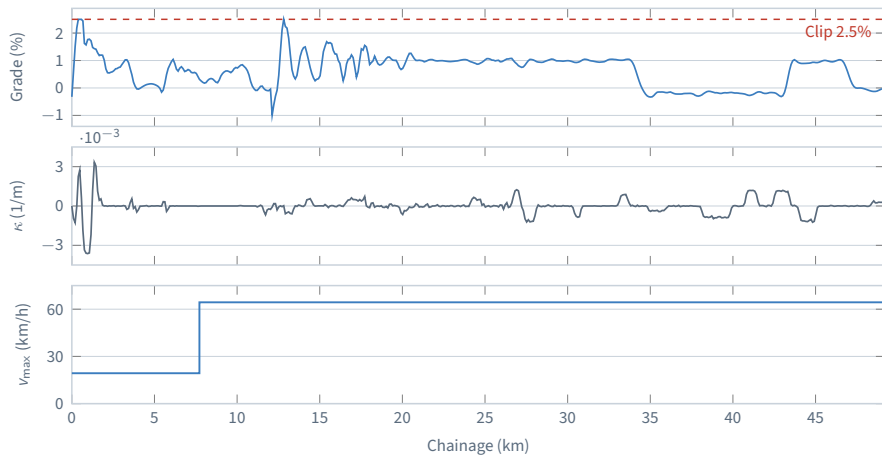
The elevation is checked visually before grade is computed. A corridor joined incorrectly, or one that jumps between parallel tracks, is obvious in 3D but **not visible in a grade plot**.

Stage 3: From Elevation to Grade



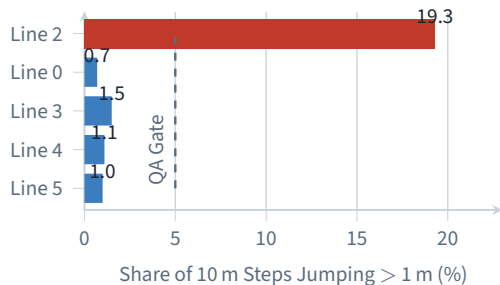
The line climbs from roughly 1430 m to 1740 m over 49 km. Errors of about a metre in the elevation data are small compared with that climb. They are large compared with the **elevation change between neighbouring 10 m samples**, which is what grade measures. The pipeline therefore corrects bridges and tunnels first, then smooths and differentiates in one step.

The Three Fields the Simulator Consumes



Line 0, after the corridor fix on the next slide. The first 15 km, through the urban area, is noisy. After that the grade stays near one percent.

Finding and Removing a Bad Corridor



The first suspected cause was too little smoothing. **That was not the cause.**

Line 2's *raw* elevation is wrong before any smoothing: 19.3% of its 10 m steps jump more than a metre. Real track cannot climb a metre every 10 m over a fifth of its length, so **these samples are not on the track.**

Smoothing cannot repair this. It only averages the incorrect values.

The fix is a quality check on the raw elevation, applied *before* smoothing. Line 2 is rejected, the smoothing window increases from 200 m to 800 m, and the grade limit drops from 4% to **2.5%**. The limit now works as an error alarm rather than a filter: 4% is far steeper than freight main lines are built, so the old limit let a broken corridor pass for months.

Results of the Fix, and a Larger Corridor Set

The Four Surviving Corridors

Corridor	rms	p95	Max
Line 0 (49 km)	0.86%	1.45%	2.50%
Line 3 (29 km)	0.72%	1.25%	1.47%
Line 4 (27 km)	0.39%	0.75%	1.23%
Line 5 (22 km)	0.58%	0.87%	1.17%

The worst overspeed falls from **67.8 to 25.7 m/s**.
The 245 km/h freight trains came from Line 2's false grades.

Runs over the limit by >5 m/s are unchanged (13.3% to 13.5%). **This has a second cause:** driving commands are generated without regard to the terrain.

A Much Larger Corridor Set

The OpenStreetMap search failed five times in a row over 4.5 hours. It was replaced by the USDOT/BTS North American Rail Network, which records how segments connect, labels yard track, and gives corridors real names (*Moffat Tunnel*, *Union Pacific*) instead of `route_line2`.

95,936

Main-Line Segments • 274,145 km

Next step: extract corridors from this network.

1. Motivation

2. Related Work

3. Formulation

4. Reference Simulator

5. Routes

6. Data & Tooling

7. Architecture

8. Training

9. Evaluation

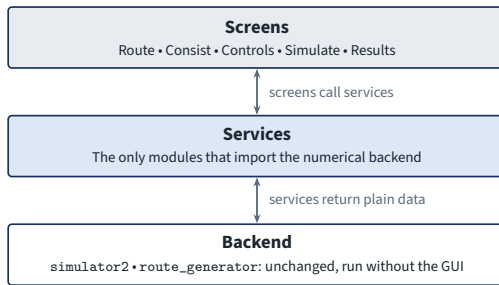
10. Control

11. Conclusion

ACT 6

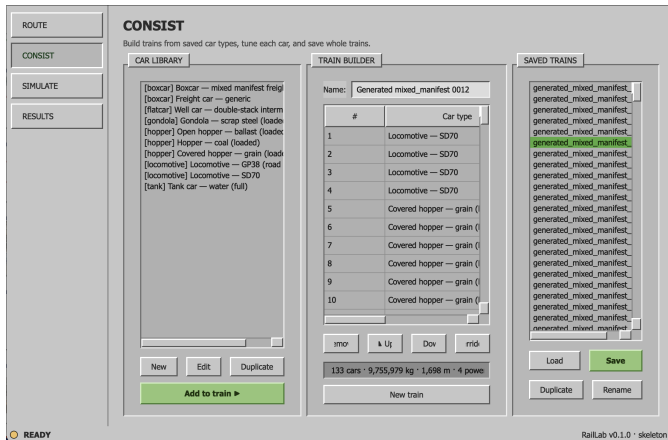
Tooling, GPU, and Data

Why a Graphical Console



Building a scenario means checking the train and its control commands **visually** before trusting them, which is easier in a graphical tool than in scripts. One design rule keeps the tool separate: **only the service layer calls the simulation code**. The simulator is unchanged by the application and still runs without it.

RailLab: Building a Consist

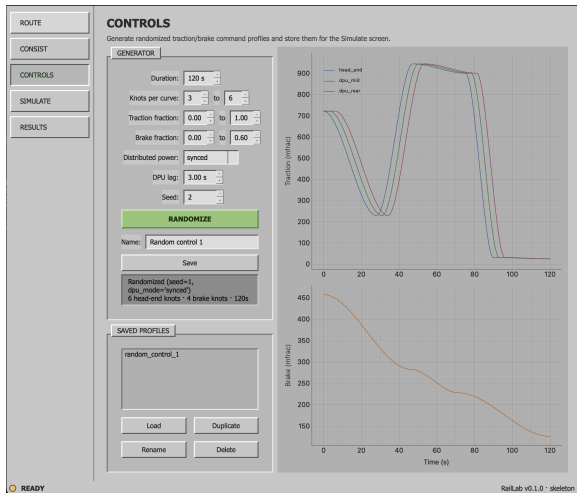


A library of reusable car types, the ordered train under construction with per-car overrides, and saved trains.

The running summary reports car count, total mass, total length and powered vehicles.

Backing it: **10 car types** and roughly **500 saved consists**.

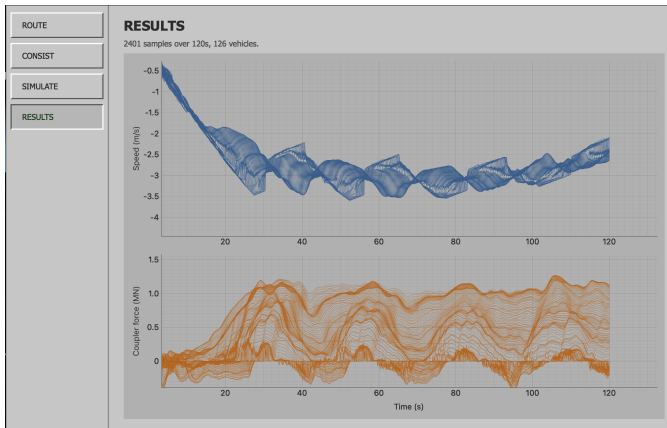
RailLab: Control Profiles and Distributed Power



Randomization bounds and distributed-power settings, with a live preview of the traction command per power role and the shared brake command.

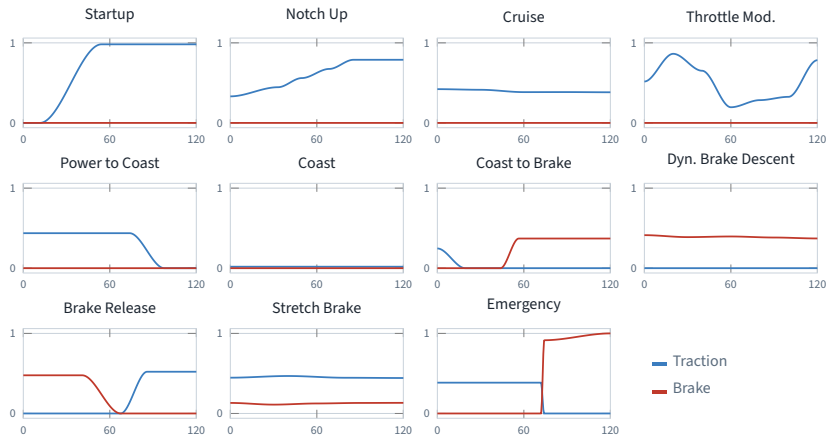
The offset between the three traction traces is the configured lag of the mid-train and rear helper units behind the head end.

RailLab: A 126-Vehicle Consist, 120 Seconds



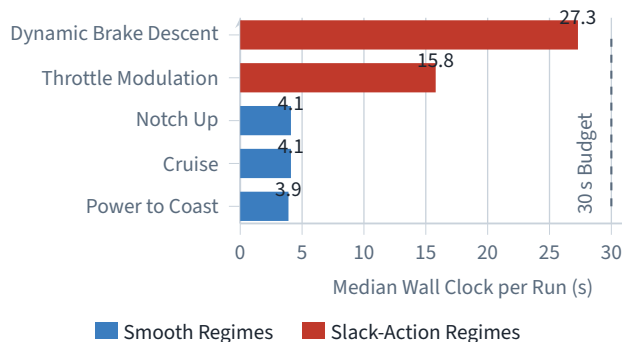
Every vehicle's speed above every coupler's force, on the same time axis. **The spread between vehicle speeds** widens and narrows as slack runs in and out, and the coupler forces below spread out at the same times.

Driving Regimes Instead of Random Commands



Fully random commands mostly describe driving no operator would do. Instead, commands come from **11 typical manoeuvres**. A check after generation ensures traction and brake are not applied together, with **0 violations across 5,500 profiles**. Two exceptions are intended: *stretch braking* applies brake against traction, and in an *emergency* the two overlap while traction fades.

The NumPy Simulator Could Not Build This Dataset



Two causes, both fixed by moving the simulator to the GPU:

1. The equations used a **Python loop** over vehicles: 130 slow iterations per evaluation at $N = 130$.
2. Force changes abruptly at the edge of the slack, so the solver shrank its step whenever *any* coupler crossed it. On a long train, that is **almost always**.

Estimated cost of a 10,000-scenario corpus: **~170 CPU-hours**.

What the GPU Port Is Required For

Required for

Data generation. Without it the two 10,000-scenario corpora at N up to 150 do not exist.

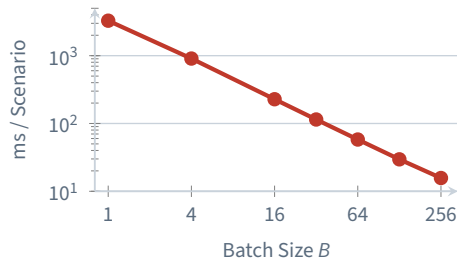
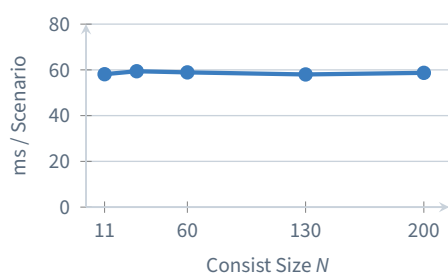
The speedup measurement. The surrogate's speedup must be measured against a fast version of the simulator, or the comparison is unfair.

Not required for

The model. An earlier design evaluated the physics equations F_{phys} in every training step. That design was dropped, and **training no longer evaluates them.**

The port also replaced the adaptive solver with fixed-step RK4. Every run now takes $\lceil \text{duration}/\text{dt} \rceil$ steps *regardless of the driving regime*, so a run can no longer stall and no timeout is needed.

Run Time Does Not Grow with Train Length



Going from 11 to 200 vehicles has no measurable effect on run time. Run time is set by the number of integration steps, each launched as one pre-recorded GPU operation, not by the amount of arithmetic. The GPU is far from fully used even at $B = 256$ scenarios and $N = 200$ vehicles.

Consequence for the thesis: the long-train test set (ood_size, $N = 150$) can now be generated, so the claim that the model works for any train length can be tested.

Numerical Precision, Measured

1.20×

Cost of double over single
precision. The plan assumed
 $\sim 64\times$

60%

Of the 0.05 m/s velocity
tolerance consumed by float32
rounding alone

f64 → f32

Integrate in double, store in
single

Run time is limited by **per-step overhead, not arithmetic**, so double precision costs little here. The original plan assumed single precision; measuring the cost reversed that choice.

Two Corpora

	data/v1	data/v2
Curvature	off	linear, $k = 1.0$
Scenarios	10,000	10,000
On disk	12.4 GB	12.4 GB
Integration	1336 s	1389 s

v2 is the dataset used for training. v1 is kept so the effect of the curvature term can be **measured** on the full dataset.

753 million

Vehicle-Steps per Corpus

~25 min

Wall Clock, Against a ~170 CPU-Hour Estimate

N 10–150 • T 721–2,397 • duration 180–599 s • 500 consists • 11 regimes • 5 corridors.

Two Storage Decisions

data/v2/	
— build_config.json	sampling ranges, git commit, seed
— index.parquet	10,000 rows $\times$ 43 columns
— norm_stats.json	train split only
— splits.json	explicit scenario ids per split
— routes/	written once per corridor, not per scenario
— scenarios/<route>/	one compressed .npz per scenario

Static and dynamic separated. H_{hist} would be $[T, N, 11]$, but seven channels never change, so they are stored once rather than at every step.

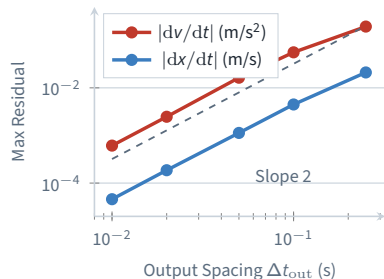
Routes referenced, not duplicated. A corridor is reused across $\sim 2,000$ scenarios.

edge_dynamic is stored even though it can be computed from the node state, so it can be used as a training target. Act VII shows why this matters.

Split	n	Basis
train	4,043	corridors line0 / line3 / line4
val	517	random 10% of eligible
test_id	496	random 10% of eligible
ood_size	955	consists flagged ood_size
ood_grade	1,999	route_line2, steepest
ood_corridor	1,990	route_line5, wholly held out
control_eval	0	not buildable; see the caveats

Corridor reservations are checked **before** the consist-level ood_size flag, so a reserved corridor cannot leak into training.

Validation, and One Finding That Changed Training



Both datasets pass three kinds of check. The most important: the equations are re-implemented separately, using *only* the scenario file and its route file, so **a bug in the original code is not repeated in the check.**

Each halving of the step size cuts the residual by $\sim 4\times$, which is second-order convergence. The residual therefore comes from the finite-difference estimate, **not** from the stored trajectory.

Consequence for training: the rate of change ds/dt should not be estimated from differences between stored states, as the schema originally suggested. At 0.25 s spacing that error is **as large as the signal the model must learn.** It is recomputed from the equations instead.

Known Data-Quality Caveats

Overspeed, data/v2

Never over the limit	72.9%
Over by > 5 m/s	17.8%
Over by > 20 m/s	2.4%
Worst	67.8 m/s

Related: **22%** of scenarios roll backward faster than 1 m/s, for the same reason.

The simulator **intentionally does not enforce** the speed limit (`route_vmax`). The limit is information for the controller, not a physical force. A coasting train on a long downgrade can keep speeding up for the full 600 s.

Overspeed follows the grade data: 39% of `route_line2` scenarios against 2% of `route_line5`. **The physics is correct but the input was flawed**; Act V describes the fix.

These runs are kept. Whether they are bad data or useful hard cases is decided at training time. Keeping only runs within 5 m/s of the limit retains $\sim 81\%$.